CHAPTER 1

INTRODUCTION

A GPS (Global Positioning System) tracker is a device that uses satellite-based navigation technology to determine and monitor the real-time location of an object, person, or vehicle. GPS trackers have become an essential tool in various industries, including transportation, logistics, security, and personal safety.

The GPS tracking system consists of three main components: satellites, ground control stations, and GPS receivers. Satellites orbiting the Earth continuously transmit signals, which are received by GPS devices. By analyzing the signals from multiple satellites, the tracker can accurately calculate its precise geographic coordinates (latitude and longitude). This information can then be transmitted to a remote monitoring system via cellular networks, GSM modules, or the internet, allowing real-time tracking and historical route analysis.

GPS trackers come in different forms, including portable personal trackers, vehicle-mounted trackers, and asset tracking devices. These devices often include additional features such as geofencing, speed alerts, and SOS buttons for emergency situations. Many GPS trackers integrate with mobile applications or web-based platforms, enabling users to access location data conveniently from smartphones or computers.

The applications of GPS tracking technology are extensive. In the transportation sector, fleet management systems use GPS trackers to optimize routes, monitor driver behavior, and enhance vehicle security. Law enforcement agencies utilize GPS tracking for surveillance and crime prevention. Personal GPS trackers provide safety for children, elderly individuals, and adventurers by allowing their locations to be tracked in real time.

In conclusion, GPS trackers have revolutionized location-based monitoring and security. With advancements in technology, modern GPS tracking devices are becoming more compact, accurate, and feature-rich, making them indispensable tools for navigation, safety, and operational efficiency.

EXECUTIVE SUMMARY

The **ESP-12E NodeMCU GPS Tracker** is an advanced and efficient tracking system that utilizes GPS technology to monitor and transmit location data in real-time. This system is designed to provide a **cost-effective, reliable, and remotely accessible solution** for tracking objects, vehicles, or individuals. The integration of the **Blynk platform** allows for seamless remote monitoring and control via a smartphone application, making it a user-friendly and accessible tracking solution.

The system is powered by the **ESP-12E NodeMCU microcontroller**, which processes GPS data received from the **Neo-6M GPS module**. The GPS module continuously tracks the location coordinates and transmits them to the microcontroller, which then sends the data to the Blynk cloud for remote access. The real-time latitude and longitude information is displayed in the Blynk app, enabling users to monitor movement and location from anywhere.

In addition to tracking capabilities, the system includes **an LED light and a buzzer**, which provide **visual and audible alerts** for various notifications. The LED can be controlled remotely via Blynk and can also indicate alerts such as geofencing breaches or unauthorized movement. Similarly, the buzzer can be triggered to generate an audible alarm when needed, enhancing security and real-time notifications.

The system operates on a **stable power supply**, typically provided through a **5V USB connection or a lithium-ion battery**, ensuring continuous functionality. For enhanced portability and field applications, a **rechargeable battery with a solar panel** can be used to make the system self-sustaining.

This **GPS tracker is ideal for multiple industries**, including **fleet management, personal security, logistics, and smart city applications**. Its ability to provide real-time location tracking, remote accessibility, and security alerts makes it a highly practical and scalable solution. The project's implementation ensures enhanced efficiency, improved security, and seamless remote monitoring, making it a valuable addition to modern tracking solutions.

1.1 SYSTEM SPECIFICATION

1. 1.1 HARDWARE COMPONENTS:

- **ESP-12E NodeMCU** (ESP8266-based)
- **Neo-6M GPS Module** (or any compatible GPS module)
- **LED** (for status indication)
- **Buzzer** (for alerts)
- **Resistors (if needed)**
- **Connecting wires**

HARDWARE DESCRIPTION:

ESP8266 (NodeMCU):

NodeMCU is an open source firmware for which open source prototyping board plans are accessible. The name "NodeMCU" joins "node" and "MCU" (miniature controller unit). The expression "NodeMCU" rigorously talking alludes to the firmware as opposed to the related advancement kits.

Both the firmware and prototyping board plans are open source.

The firmware utilizes the Lua prearranging language. The firmware depends on the eLua project, and based on the Espressif Non-OS SDK for ESP8266. It utilizes many open source projects, for example, lua-cjson and SPIFFS. Due to asset limitations, clients need to choose the modules pertinent for their venture and assemble a firmware customized to their requirements. Backing for the 32-bit ESP32 has likewise been carried out.

Figure 1.1:NodeMCU

The prototyping equipment ordinarily utilized is a circuit board working as a double in-line package (DIP) which coordinates a USB regulator with a more modest surface-mounted board containing the MCU and recieving wire. The decision of the DIP design considers simple prototyping on breadboards. The plan was at first dependent on the ESP-12 module of the ESP8266, which is a Wi-Fi SoC incorporated with a Tensilica Xtensa LX106 core, broadly utilized in IoT applications

So it's simpler to utilize NodeMCU for the applications where it need less GPIO pins and furthermore needs to associate with WiFi network.

Advantages of NodeMCU:

- Affordable
- Integrated support for WIFI network.
- Reduced size of the board.
- Low energy consumption.

Neo-6M GPS Module:

One of the global positioning system (GPS) gadgets ultilizes information from satellites to find a particular point on the Earth in an interaction named trilateration. In the process, a GPS receiver estimates the distances to satellites utilizing radio signs to trilaterate. What's more, trilateration is like triangulation, which estimates points, portrayed in this outline . GPS modules contain minute processors and recieving wires that straightforwardly get information sent by satellites through devoted RF frequencies. From that point, it'll get timestamp from each noticeable satellites, alongside different bits of information. In the event that the module's recieving wire can spot at least 4 satellites, it's ready to precisely compute its position and time.

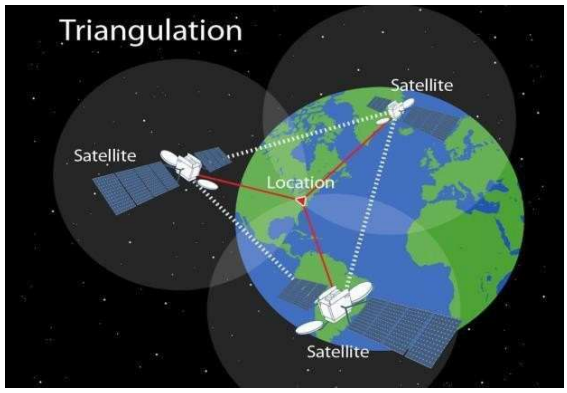


Figure 1.2 satellites points

This is a GPS module that depends on the NEO 6M GPS. This unit utilizes the latest down the line innovation to give the most ideal situating data and incorporates a bigger inherent 25 x 25mm dynamic GPS radio wire with a UART TTL attachment. A battery is additionally included with the goal that you can get a GPS lock quicker. This is a refreshed GPS module that can be utilized with ardupilot mega v2. This GPS module gives the most ideal position data, considering better execution with your Ardupilot or other Multirotor control stage.

Figure 1.3. Neo-6M GPS Module

The GPS module has sequential TTL yield, it has four pins: TX, RX, VCC, and GND.

It can track up to 22 satellites on 50 stations and accomplishes the business' most significant level of affectability for example - 161 dB tracking, while at the same time just

45mA current. In contrast to different GPS modules, it can do up to 5 locations updates a second with 2.5m Horizontal position precision. The u- blox 6 situating motor likewise flaunts a Time-To-First-Fix (TTFF) of under 1 second.

Probably the best element the chip gives is Power Save Mode(PSM). It permits a decrease in framework power utilization by specifically turning portions of the collector ON and OFF. This drastically diminishes power utilization of the module to simply 11mA making it appropriate for power touchy applications like GPS wristwatch.

The fundamental information pins of NEO-6M GPS chip are broken out to a 0.1″ pitch headers. This incorporates pins needed for correspondence with a microcontroller over UART. The module upholds baud rate from 4800bps to 230400bps with default baud of 9600.

There is an LED on the NEO-6M GPS Module which indicates the status of Position Fix. It'll blink at various rates depending on what state it's in:

- No Blinking – It's searching for satellites.
- Blink every 1s – Position Fix is found(The module can see enough satellites).

LED:

The LED light in the ESP-12E NodeMCU GPS tracker serves as a visual indicator for various status updates and alerts. It is connected to **GPIO5 (D1 pin)** and can be controlled remotely via the **Blynk app** using a button assigned to **Virtual Pin V4**. The LED turns **ON/OFF** based on user input, providing a simple yet effective way to indicate the tracker's operational status. Additionally, the LED can be programmed to function as an **automatic alert system** based on GPS data. For instance, if the GPS tracker moves outside a predefined area (geofencing), the LED can blink as a warning signal. In normal operation, the LED remains **OFF** until activated, while in emergency situations, such as unauthorized movement, it can be programmed to **blink rapidly**. This feature enhances security by providing immediate visual feedback about the device's location status. The combination of manual control via **Blynk** and automatic activation based on GPS movement makes the LED a crucial component in the smart GPS tracking system.

Figure 1.4 LED LIGHT

Buzzer:

The **buzzer** in the ESP-12E NodeMCU GPS tracker acts as an **audible alert system**, providing sound notifications for various events and warnings. It is connected to **GPIO4 (D2 pin)** and can be controlled remotely using the **Blynk app** via a button assigned to **Virtual Pin V4**. When activated, the buzzer emits a sound to indicate critical alerts, such as unauthorized movement or a security breach. Additionally, the buzzer can be programmed to work alongside the **GPS module**, triggering an alarm if the tracker moves beyond a predefined geofencing area. This feature enhances security by providing an instant audio warning, making it useful for **anti-theft applications** and **emergency alerts**. The buzzer can also be activated manually by the user through the **Blynk app**, allowing remote control of alerts. Its combination of **automatic activation based on GPS movement** and **manual control via Blynk** makes it a vital component in improving situational awareness and ensuring real-time notifications in the smart tracking system.

Figure 1.5 BUZZER

Power Supply:

The **power supply** for the ESP-12E NodeMCU GPS tracker is a crucial component that ensures the reliable operation of the system. The NodeMCU operates on a **3.3V logic level**, but it is typically powered using a **5V supply**, which can be provided through a **USB connection**, a **Li-ion battery**, or a **regulated power adapter**.

1.1.2 SOFTWARE SPECIFICATION

Operating System	: Windows 10 pro
Programming Language	: C/C++ (Embedded Programming)
Application	: Arduino IDE (C++ Framework)
Mobile Application	**:** Blynk Mobile App
Cloud	: Blynk Cloud

SOFTWARE DESCRIPTION:

Arduino IDE:

The **Arduino IDE (Integrated Development Environment)** is an open-source software used for writing, compiling, and uploading code to Arduino boards. It provides a simple and user-friendly interface with a built-in code editor, message console, and a serial monitor for debugging. The IDE supports **C and C++** programming languages and includes a vast library of pre-written functions that simplify hardware interaction. It allows users to upload code to microcontrollers via **USB** and is compatible with various operating systems, including Windows, macOS, and Linux. Additionally, it supports third-party boards like **ESP32 and NodeMCU**, making it a versatile tool for IoT and embedded systems development.

The interface of the Arduino IDE includes a text editor for writing code (known as "sketches"), a compiler to convert the code into machine language, and a serial monitor to communicate with the Arduino board. The IDE is compatible with a wide range of Arduino boards, such as the Arduino Uno, Mega, Nano, and others, and it allows users to easily upload code to the microcontroller via USB or serial connections.

Code Editor:

A simple text editor with syntax highlighting to write and edit Arduino programs.

Libraries:

It supports a wide array of pre-built libraries that simplify the integration of sensors, actuators, and communication modules.

Serial Monitor:

A tool for debugging, allowing the user to receive real-time data from the board or send commands to it.

Cross-platform:

The IDE is available for Windows, macOS, and Linux, making it highly versatile for different environments.

1.Open Ardunio IDE as shown below:

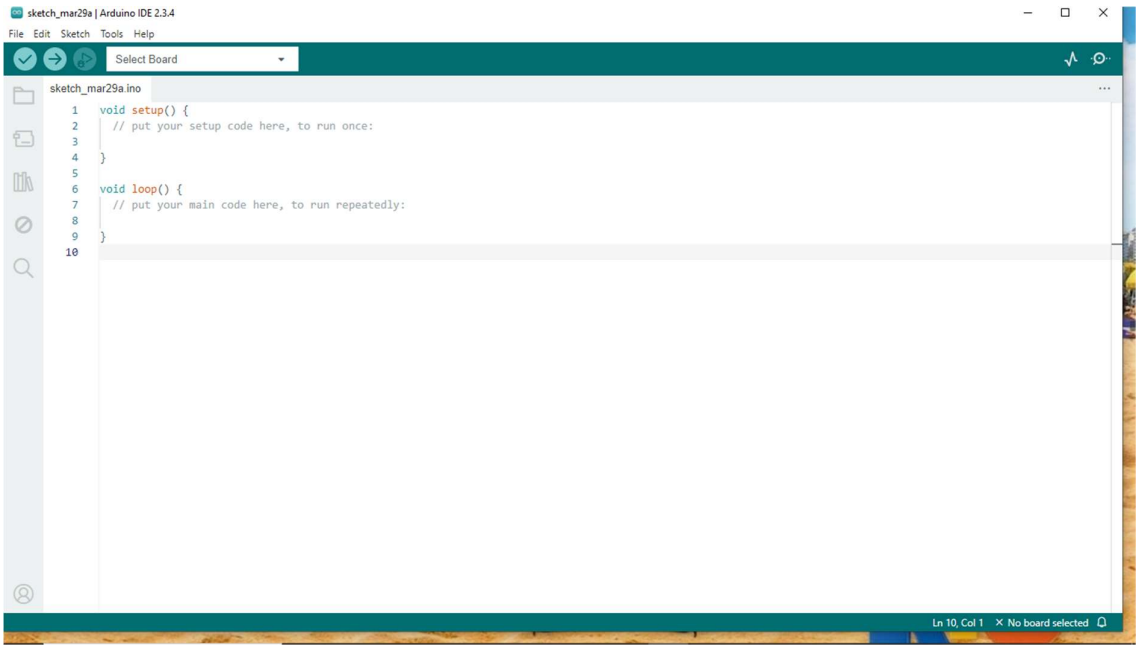


Figure1.6 Ardunio IDE

2. Select the Board and COM port:

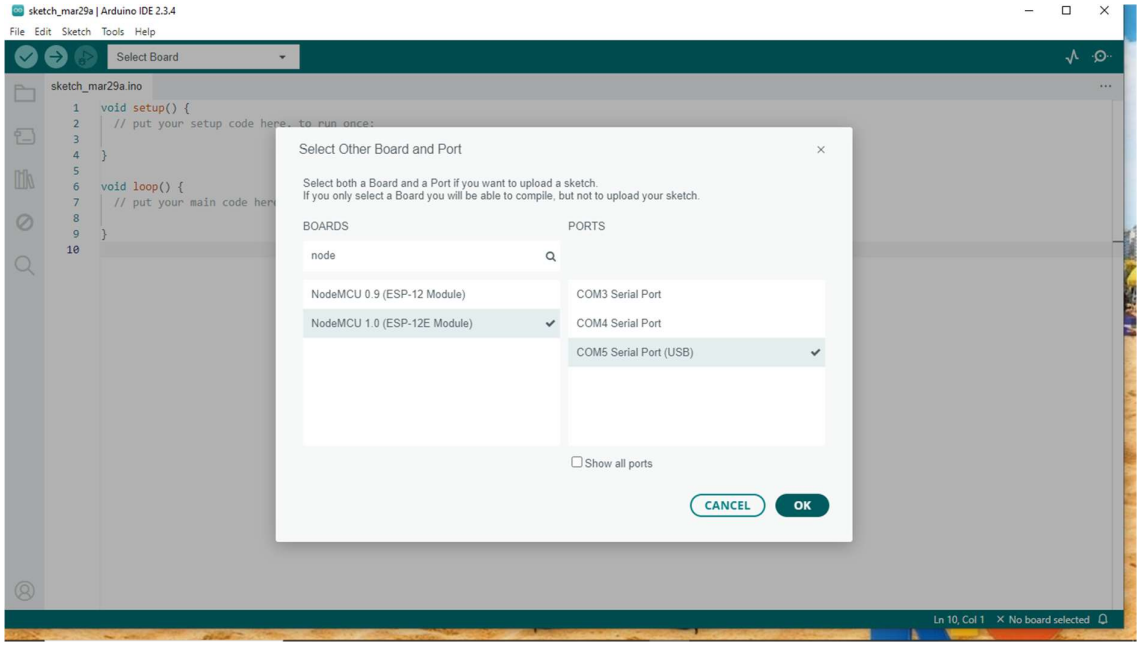


Figure 1.7 Select the Board and COM port

3. Write the sketch in Arduino IDE as shown below

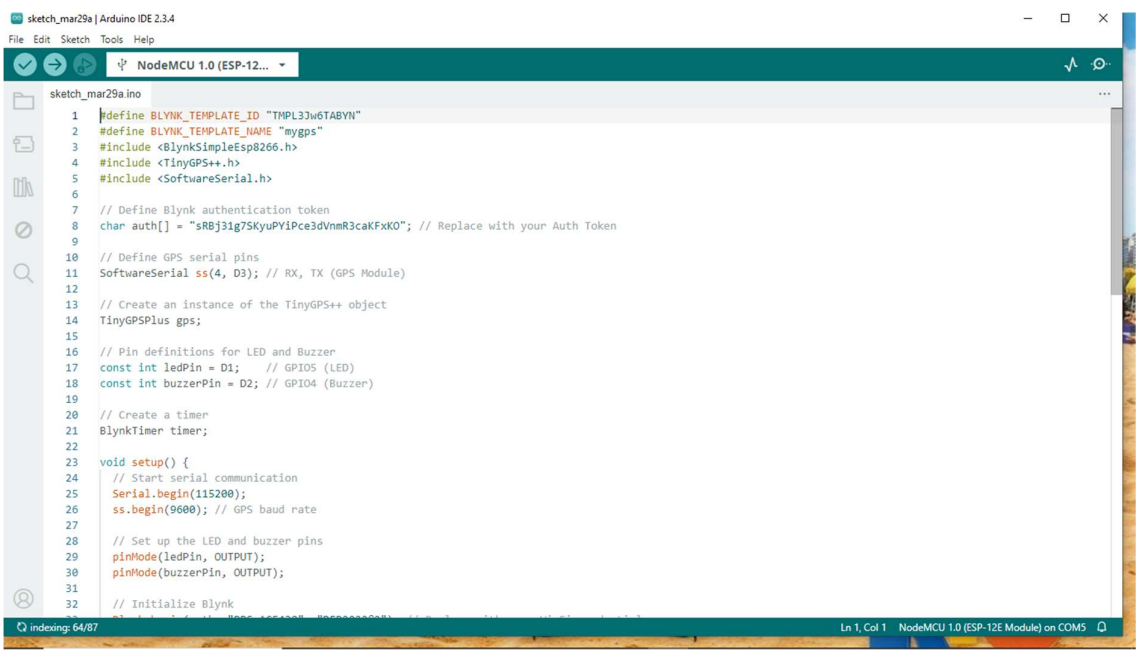


Figure 1.8 Write the sketch in Arduino IDE

4. Compile and upload the sketch in the Arduino as shown below:

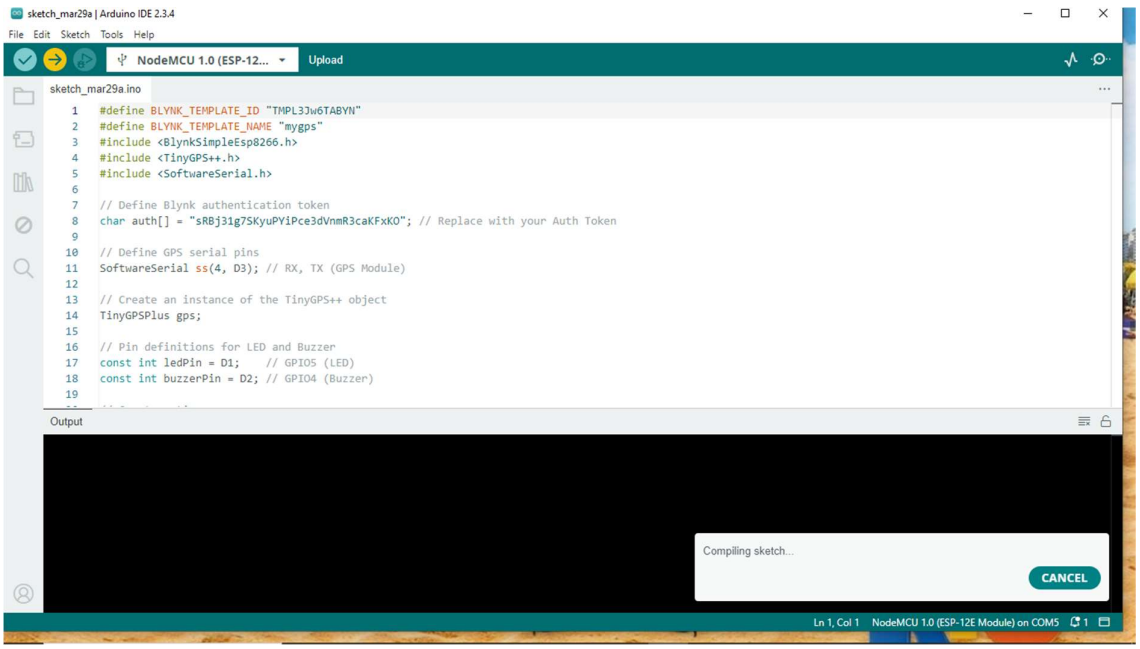


Figure 1.9 Compile and upload the sketch in the Arduino

INPUT AND OUTPUT

Input:

The system receives input from various hardware components and software interactions:

1. **GPS Data (Neo-6M GPS Module)** – Captures **real-time latitude and longitude** coordinates.
2. **Wi-Fi Connectivity (ESP8266 NodeMCU)** – Receives **network credentials** for connecting to **Blynk Cloud**.
3. **User Commands (Blynk Mobile App)** – Users can send commands to:
   - Activate or deactivate the **LED and Buzzer** for alerts.
   - Request **real-time location updates**.
4. **Google Maps API** – Processes GPS data to generate a **real-time tracking link**

Output:

The system provides the following outputs:

1. **Live GPS Location on Blynk Mobile App** – Displays **real-time latitude and longitude** coordinates.
2. **Google Maps Integration** – Generates a **tracking link** for location visualization.
3. **LED and Buzzer Alerts** – Provide **visual and audio notifications** based on remote user commands.
4. **Data Transmission to Blynk Cloud** – Stores and processes location data for remote access.

This **input-output system** enables real-time tracking, cloud-based monitoring, and remote control via a **mobile app**, making it a **cost-effective and efficient IoT solution**.

Embedded C/C++ Programming

C and C++ are widely used in embedded programming due to their efficiency, direct hardware access, and portability. C is preferred for low-level programming, while C++ is used in complex systems requiring modularity. Embedded systems rely on memory management, bitwise operations, interrupts, and communication protocols like I2C, SPI, and UART. Real-time constraints make efficient coding crucial, avoiding dynamic memory allocation and optimizing power usage. Microcontrollers, IoT devices, and industrial automation systems benefit from C/C++ for performance and reliability. Mastering these languages is key to developing robust embedded applications.

CHAPTER 2

SYSTEM STUDY

2.1 EXISTING SYSTEM

2.1.1 DESCRIPTION OF THE EXISTING SYSTEM

The existing system for **construction and heavy equipment tracking** primarily relies on manual methods, traditional GPS trackers, and expensive fleet management solutions. Many companies still use **manual logs and paper records**, which lead to errors, delays, and difficulties in retrieving information when needed. Some businesses have adopted **basic GPS tracking devices**, but these often lack **real-time alerts and remote control features**, making them inefficient for security and monitoring. Large construction firms may invest in **commercial fleet management systems**, but these solutions are **costly and complex**, making them unsuitable for smaller companies.

A major challenge in the industry is **equipment theft**, which results in billions of dollars in losses annually. The current systems fail to provide **immediate alerts** when machinery is moved without authorization, increasing the risk of theft. Additionally, **maintenance monitoring is inefficient**, as most companies lack automated tracking for **engine hours, fuel consumption, or service schedules**, leading to unexpected breakdowns and costly repairs. Overall, the existing tracking solutions **lack integration with IoT and cloud platforms**, making them **costly, inefficient, and unreliable for real-time monitoring and security**.

2.1.2 DRAWBACKS OF THE EXISTING SYSTEM

The current **construction and heavy equipment tracking** system has several limitations that affect efficiency, security, and operational costs.

1. **No Real-Time Tracking** – Many systems rely on **manual logs or basic GPS tracking**, which do not provide **instant updates** on equipment location and movement.
2. **High Risk of Theft** – The absence of **geofencing and instant alerts** makes it easy for thieves to move equipment undetected, leading to **huge financial losses**.

3. **Expensive and Inaccessible Solutions** – Advanced fleet management systems are **too costly** for small and mid-sized businesses, making **affordable tracking solutions** necessary.
4. **No Automated Alerts** – Traditional systems do not include **buzzer alarms or LED notifications** for unauthorized movement, delaying response times during emergencies.
5. **Inefficient Maintenance Tracking** – There is no **automated tracking of engine hours, fuel levels, or servicing schedules**, which can result in **unexpected breakdowns** and costly repairs.
6. **Manual Data Entry & Errors** – Many companies still rely on **paper logs or spreadsheets**, leading to **human errors, data loss, and inefficiencies** in tracking.
7. **Lack of IoT & Cloud Integration** – Existing systems do not support **remote monitoring via mobile apps**, making it difficult to manage equipment from anywhere.

2.2 PROPOSED SYSTEM

2.2.1 DESCRIPTION OF PROPOSED SYSTEM

The proposed system is an **IoT-based GPS tracking solution** that uses an **ESP-12E NodeMCU**, a **Neo-6M GPS module**, and the **Blynk platform** to provide **real-time location tracking** along with **remote monitoring and security alerts**. This system is designed to overcome the limitations of traditional tracking methods by offering **live GPS updates, automated alerts, and remote access via a mobile app**.

The system continuously collects **latitude and longitude data** from the **GPS module** and transmits it to the **Blynk cloud** using Wi-Fi. Users can view the real-time location of tracked equipment directly from the **Blynk mobile app**. Additionally, the system integrates an **LED and a buzzer** to provide **instant alerts** when unauthorized movement is detected. Through **Blynk's virtual pins**, users can remotely control the buzzer and LED to signal warnings or alerts.

Furthermore, the system can be **enhanced with geofencing** to trigger alerts when the tracked object moves outside a predefined boundary. It also has the potential to integrate with a **GSM module (SIM900A)** to send **SMS alerts** in case of emergencies or lost Wi-Fi connectivity.

The proposed system is **cost-effective, efficient, and scalable**, making it ideal for **construction equipment tracking, vehicle monitoring, and security applications**

2.2.2 FEATURES OF THE PROPOSED SYSTEM

The **ESP-12E NodeMCU GPS Tracker with Blynk Integration** is designed to provide **real-time tracking, remote monitoring, and security alerts**. Below are its key features:

1. *Real-Time GPS Tracking*
2. *Remote Monitoring via Blynk App*
3. *LED & Buzzer Alerts for Security*
4. *Geofencing for Unauthorized Movement Detection*
5. *Low-Cost & IoT-Based Solution*
6. *Expandable with GSM & Battery Backup* (Optional)
7. *Secure & Scalable System*

CHAPTER 3

SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

File design refers to the structure and organization of data stored within a system to ensure efficient data retrieval, processing, and management. In the **ESP-12E NodeMCU GPS Tracker**, file design plays a crucial role in handling location data, system logs, and user inputs efficiently. The system may use **local storage** (such as EEPROM or an SD card) or **cloud storage** (through the Blynk platform) to manage GPS coordinates, timestamps, and alert statuses.

The key aspects of file design in this project include **data format**, **storage method**, and **retrieval process**. GPS location data, including **latitude, longitude, and timestamps**, can be stored in a structured format like **CSV or JSON** for easy processing. The Blynk cloud acts as a remote database, storing real-time tracking information and providing seamless access via the mobile application. Additionally, the system logs **buzzer and LED activation statuses** to track security events and alerts. A well-structured file design ensures optimal system performance, fast data retrieval, and efficient remote monitoring.

3.2 INPUT DESIGN

The input design of the **ESP-12E NodeMCU GPS Tracker** ensures efficient data collection and processing for real-time tracking. The main inputs of the system include:

1. GPS Data Input:

- **Neo-6M GPS Module** provides real-time **latitude and longitude coordinates**.
- Data is received through the **SoftwareSerial interface** and processed by the **ESP-12E NodeMCU**.
- GPS data is sent to the **Blynk cloud platform** for remote monitoring.

2. User Inputs via Blynk App:

- Users can manually control system features through **Blynk virtual buttons**.

- **Virtual Pin V3**: Displays the **GPS location on the Blynk** for real-time tracking.
- **Virtual Pin V4**: Controls the **Buzzer** (ON/OFF for alerts).
- **Virtual Pin V5**: Controls the **LED** (ON/OFF for status indication).
- User inputs are processed by the NodeMCU and executed in real time.

3. Network Input:

- **Wi-Fi connection** is used for real-time data transmission to Blynk.
- If Wi-Fi is unavailable, an **optional GSM module (SIM900A)** allows input commands via **SMS**.

The well-structured input design ensures **accurate GPS tracking, user control, and reliable communication**, making it ideal for **vehicle tracking, security alerts, and remote monitoring applications**.

3.3 OUTPUT DESIGN

The **Output Design** of the **ESP-12E NodeMCU GPS Tracker** focuses on effectively presenting real-time tracking data and system alerts to users. The system ensures that GPS coordinates, status indicators, and alerts are displayed in a clear and accessible manner. The key outputs of the system include:

1. GPS Data Output:

- The **latitude and longitude coordinates** are sent to the **Blynk cloud platform**.
- **Virtual Pin V3**: Displays the **GPS location on the Blynk Map Widget**, allowing users to view real-time tracking data.
- The GPS coordinates are also printed on the **serial monitor** for debugging purposes.

2. Alert and Status Indicators:

- **Virtual Pin V4**: Controls the **Buzzer** (ON/OFF), providing an audible alert in case of security events or movement detection.
- **Virtual Pin V5**: Controls the **LED** (ON/OFF) to indicate system status or security warnings.
- Alerts can be triggered based on geofencing or predefined conditions.

3. Communication and Notifications:

- **Blynk App Notifications**: Sends real-time updates when the tracker moves or crosses predefined boundaries.
- **SMS Alerts (Optional)**: If a **GSM module (SIM900A)** is integrated, SMS notifications can be sent in case of movement or emergency alerts.
- The system ensures **fast and accurate output delivery** to keep users informed in real time.

This output design ensures **efficient monitoring, clear alerts, and reliable communication**, making the system ideal for **vehicle tracking, security applications, and remote asset management**.

3.4 CODE DESIGN

The programming language used for the **ESP-12E NodeMCU GPS Tracker** is **C++ (Arduino language)**. The code follows the **Arduino framework**, which is based on C/C++ and is used to program microcontrollers like the ESP8266 and ESP32.

Key features of the **Arduino-based C++ programming** used in this project include:

- **Library Integration**: Uses libraries like BlynkSimpleEsp8266.h, TinyGPS++.h, and SoftwareSerial.h for communication and data processing.
- **Setup and Loop Structure**: Uses the standard setup() and loop() functions for initialization and continuous operation.
- **Hardware Control**: Implements functions to read GPS data, send location updates, and control external components like LEDs and buzzers.
- **Networking**: Establishes Wi-Fi communication using Blynk.begin().
- **Event Handling**: Uses BLYNK_WRITE() functions to respond to user inputs from the Blynk app.

This **C++ (Arduino-based)** programming approach ensures efficient, real-time operation of the **GPS tracking system**.

3.5 SYSTEM DEVELOPMENT

3.5.1 DESCRIPTION OF MODULES

1. **Design Phase:**

The **Design Phase** of the ESP-12E NodeMCU GPS Tracker focuses on creating a structured plan for **hardware integration, software architecture, and system flow**. This phase ensures that the system operates efficiently and meets the requirements for **real-time GPS tracking, remote monitoring, and alert notifications**. The design phase consists of the following key components:

SYSTEM ARCHITECTURE

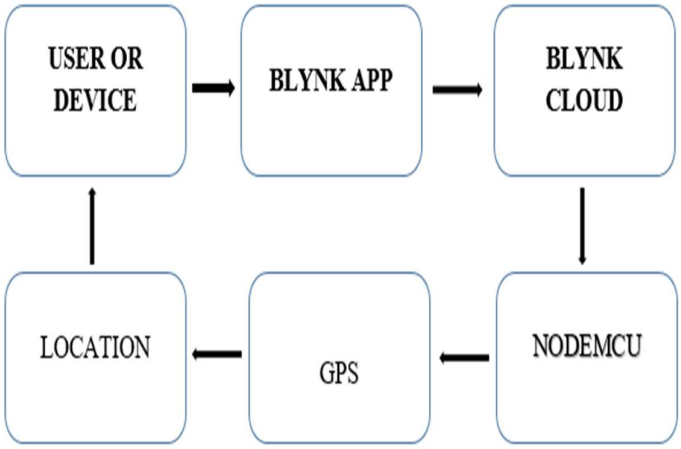


Figure 3.1 SYSTEM ARCHITECTURE

DATA FLOW DIAGRAM:

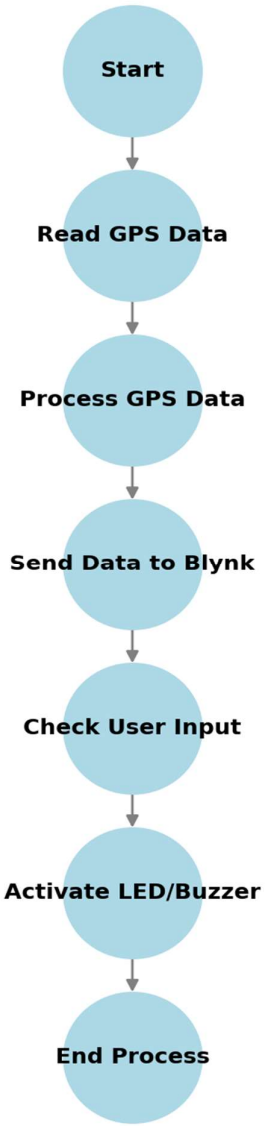


Figure 3.2 Data Flow Diagram (DFD) Level 1

2. Implementation Phase:

The **Implementation Phase** involves assembling hardware, developing software, testing, and deploying the ESP-12E NodeMCU GPS Tracker. The ESP-12E connects with the **Neo-6M GPS module** for real-time location tracking, while a **buzzer and LED** provide alerts. The system processes GPS data, updates the **Blynk Map Widget (V3)**, and allows users to

control alerts remotely via the **Blynk app**. After rigorous testing for accuracy and functionality, the system is deployed for **vehicle tracking, asset security, and remote monitoring**, ensuring reliable performance and real-time updates.

MODULES

1. GPS Data Acquisition Module

This module is responsible for collecting real-time **latitude and longitude** from the **Neo-6M GPS module**. The GPS module communicates with the **ESP-12E NodeMCU** using serial communication (**RX/TX pins**). The **TinyGPS++** library is used to decode and process the raw GPS signals into meaningful location data. The system continuously reads and updates GPS data to ensure accurate tracking.

Key Functions:

- Captures real-time GPS coordinates (latitude & longitude).
- Filters and processes GPS signals using the TinyGPS++ library.
- Ensures stable communication with the ESP-12E via **SoftwareSerial**.

2. Data Processing & Transmission Module

Once the GPS data is received, this module **processes** it and **sends** it to the Blynk cloud using **Wi-Fi communication**. The ESP-12E connects to a **Wi-Fi network**, ensuring the data is transmitted in real time to the **Blynk App** for remote monitoring.

Key Functions:

- Reads and extracts location data from the GPS module.
- Connects to Wi-Fi and establishes communication with the **Blynk cloud**.
- Sends processed GPS data to the **Blynk server** for real-time tracking.

3. Blynk Integration & User Interface Module

This module enables **real-time GPS tracking and user interaction** via the **Blynk app**. The location data is sent to the **Blynk Map Widget (V3)**, allowing users to visualize the

tracker's position on a map. The user can also **interact with the system** through virtual buttons to control alerts like the **LED and buzzer**.

Key Functions:

- Displays real-time GPS location on **Blynk Map (V3)**.
- Allows users to monitor GPS data remotely via the **Blynk app**.
- Provides an interface for users to control the **LED (V5)** and **Buzzer (V4)**.

4. Alert & Notification Module

This module is responsible for triggering **alerts** based on user input or predefined conditions. Users can remotely control an **LED and buzzer** using the **Blynk app**, which can be used to signal specific alerts. Additionally, the system can be configured to send alerts if the device moves outside a defined **geofencing** boundary.

Key Functions:

- Controls **LED (V5)** and **Buzzer (V4)** for alerts.
- Notifies the user in case of unexpected movement.
- Can implement **geofencing** to trigger alerts when out of a set location.

5. **Power Management Module**

Since the ESP-12E operates on **3.3V**, this module ensures a **stable power supply** to prevent voltage fluctuations. A regulated **5V-3.3V power source** is used to power the ESP-12E, GPS module, and other connected components.

Key Functions:

- Converts **5V to 3.3V** for ESP-12E and GPS module operation.
- Ensures stable power supply for continuous tracking.
- Optimizes power consumption for better efficiency.

3. Integration Phase:

The **Integration Phase** of the ESP-12E NodeMCU GPS Tracker involves combining all individual modules and ensuring they work together as a unified system. This phase focuses on **hardware integration, software integration, communication between components, and system validation** to guarantee smooth functionality and accurate GPS tracking.

4. Development Phase:

The **Development Phase** involves assembling hardware, coding firmware, and testing the ESP-12E GPS Tracker. The **ESP-12E, GPS module, LED, and buzzer** are connected, and firmware is developed using **Arduino IDE, TinyGPS++, and BlynkSimpleEsp8266**. The system integrates **Wi-Fi and Blynk** for real-time tracking and remote alerts. After testing **GPS accuracy, connectivity, and user controls**, the tracker is optimized for deployment.

5. Maintenance & Support

The **Maintenance and Support** phase ensures the long-term reliability and efficiency of the ESP-12E GPS Tracker. Regular **firmware updates** help improve performance, fix bugs, and enhance security. Hardware maintenance involves checking the **ESP-12E, GPS module, LED, and buzzer** to ensure proper functionality and stable power supply. **Network monitoring** is crucial to maintaining reliable **Wi-Fi connectivity** and real-time tracking on the **Blynk app**. Additionally, user support is provided through troubleshooting assistance, documentation, and updates to ensure seamless operation. This phase helps keep the system functional, accurate, and up-to-date for long-term use.

CHAPTER 4

TESTING AND IMPLEMENTATION

The **Testing and Implementation** phase ensures the ESP-12E GPS Tracker functions correctly before deployment. **Testing** involves checking **GPS accuracy, Wi-Fi connectivity, and real-time data transmission** using the **Blynk app**. The system undergoes multiple tests to verify the correct operation of the **GPS module, LED, and buzzer**, ensuring seamless integration. **Implementation** includes deploying the tracker in real-world conditions, monitoring its performance, and making necessary adjustments for reliability. This phase ensures the system is **fully functional, user-friendly, and ready for practical applications** like vehicle tracking and asset monitoring.

1. Testing Phase:

This phase involves systematically verifying the functionality of both **hardware and software** components.

- **Unit Testing** – Each module, including the **GPS module, ESP-12E, LED, and buzzer**, is tested individually to ensure correct operation.
- **Integration Testing** – The interaction between the GPS module, ESP-12E, and the **Blynk platform** is tested to ensure seamless data transmission.
- **Functionality Testing** – The system is tested for **GPS accuracy, Wi-Fi connectivity, real-time location updates, and remote alerts**.
- **Performance Testing** – The system is run under different conditions to evaluate **response time, power efficiency, and network stability**.
- **User Acceptance Testing (UAT)** – Final testing is conducted to ensure the system meets user expectations and operates effectively.

2. Implementation Phase

After successful testing, the system is **deployed in real-world conditions** for practical use.

- **Deployment** – The GPS tracker is installed and tested in actual environments, such as **vehicles or assets**, to monitor its real-time performance.

- **Monitoring and Optimization** – The system's **GPS accuracy, connectivity, and user controls** are continuously monitored to improve performance.
- **User Training & Support** – End users are guided on how to use the **Blynk app, view GPS data, and control alerts like the LED and buzzer**.
- **Maintenance Plan** – A strategy is set up for future **firmware updates, troubleshooting, and long-term system reliability**.

Activities:

- **System Initialization** – ESP-12E connects to **Wi-Fi**, initializes **GPS module, LED, and buzzer**.
- **GPS Data Acquisition** – Neo-6M GPS module fetches **latitude and longitude**.
- **Data Processing** – TinyGPS++ library processes the GPS data.
- **Blynk Data Transmission** – Sends location data to **Blynk Virtual Pins (V1, V2, V3)**.
- **User Monitoring** – Users track **real-time GPS location** via the Blynk app.
- **LED & Buzzer Control** – Users turn **LED (V4) & buzzer (V5)** on/off remotely via Blynk.
- **Alerts & Notifications** – The system updates location continuously and alerts users.
- **System Maintenance** – Regular monitoring for **firmware updates & troubleshooting**.

This ensures **seamless GPS tracking, remote monitoring, and efficient user control.**

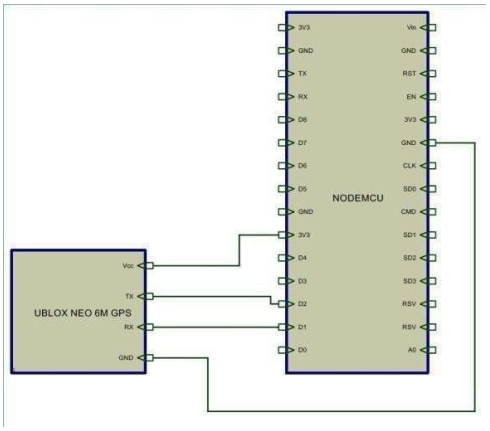


Figure 4. Nodemcu To GPS Module Connection

CHAPTER 5

CONCLUSION

The development of the **ESP-12E NodeMCU GPS Tracker** successfully integrates **real-time GPS tracking, remote monitoring, and alert functionalities** using the **Neo-6M GPS module, ESP-12E, LED, buzzer, and the Blynk platform**. The system ensures accurate **location tracking**, seamless **Wi-Fi connectivity**, and user-friendly control via the **Blynk app**, making it a practical solution for applications such as **vehicle tracking, asset management, and personal safety**.

Throughout the **development, testing, and implementation phases**, the hardware and software components were carefully integrated and optimized for **efficient performance**. The **TinyGPS++** library processes GPS data, while **BlynkSimpleEsp8266** enables real-time updates to the Blynk platform, ensuring smooth communication between the tracker and the user. The inclusion of **LED and buzzer alerts** enhances functionality by providing **real-time notifications** in case of emergencies or specific tracking conditions.

Rigorous **testing and validation** confirmed the system's **GPS accuracy, network stability, and responsiveness** to user commands. The tracker was deployed in real-world conditions, ensuring it meets reliability and performance expectations. A well-defined **maintenance and support** plan was established to provide **firmware updates, troubleshooting, and long-term stability**.

In conclusion, the **ESP-12E GPS Tracker** offers a **cost-effective, reliable, and efficient** solution for **real-time location tracking and remote monitoring**. With its flexible and scalable design, it can be further enhanced with additional sensors, cloud storage, and advanced security features to cater to diverse tracking needs. The successful implementation of this project demonstrates its potential for practical applications, providing **enhanced safety, security, and convenience** for users.

CHAPTER 6

BIBLIOGRAPHY

1. **Arduino Documentation** – Official reference for ESP-12E NodeMCU, programming, and hardware integration.

   Available at: https://www.arduino.cc

2. **ESP8266 NodeMCU Guide** – Technical documentation and tutorials on ESP-12E for IoT applications.

   Available at: https://www.espressif.com

3. **TinyGPS++ Library** – A library for parsing GPS data from the Neo-6M module.

   Available at: https://github.com/mikalhart/TinyGPSPlus

4. **Blynk IoT Platform** – Official guide on integrating IoT devices with the Blynk app for remote monitoring.

   Available at: https://www.blynk.io

5. **Neo-6M GPS Module Datasheet** – Technical specifications and interfacing details for the GPS module.

   Available at: https://www.u-blox.com

6. **SoftwareSerial Library** – Arduino library for serial communication with GPS modules and other peripherals.

   Available at:https://www.arduino.cc/en/Reference/SoftwareSerial

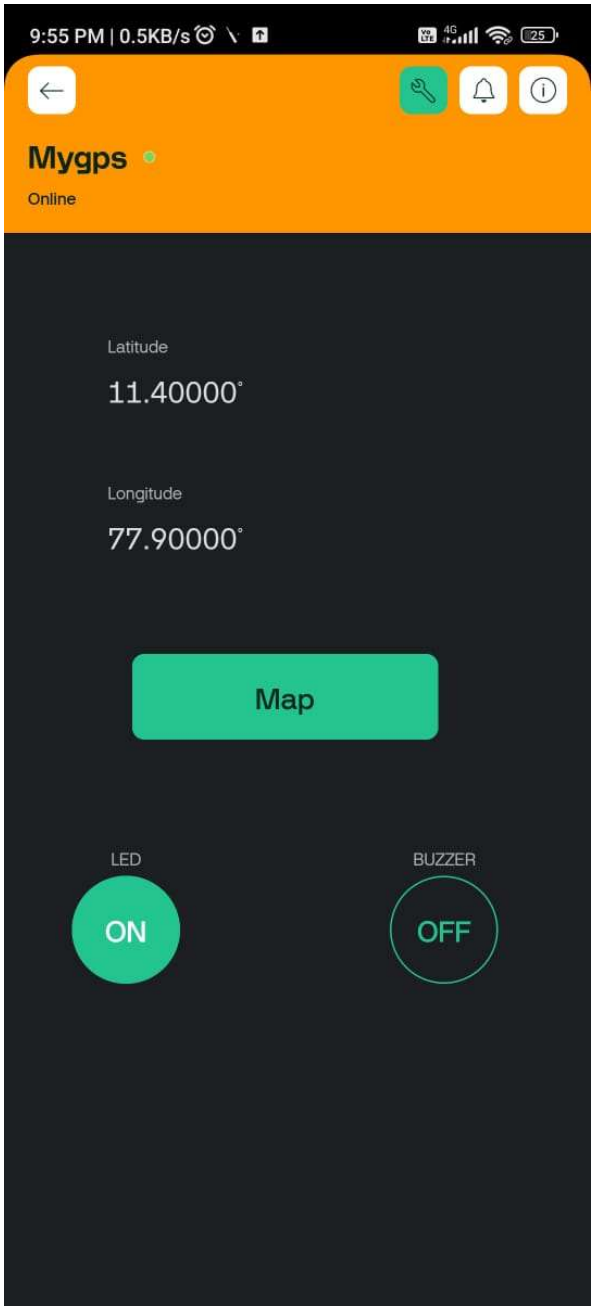
7. **IoT-Based GPS Tracking Systems Research Papers** – Various scholarly articles on GPS tracking technology and IoT integration. Sources include IEEE Xplore and ResearchGate.

SAMPLE CODE

```
#define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID"

#define BLYNK_TEMPLATE_NAME "GPS Tracker"


#include <BlynkSimpleEsp8266.h>

#include <TinyGPS++.h>

#include <SoftwareSerial.h>


// Blynk Authentication

char auth[] = "YOUR_BLYNK_AUTH_TOKEN";

char ssid[] = "YOUR_WIFI_SSID";

char pass[] = "YOUR_WIFI_PASSWORD";


// Define GPS Serial Pins

SoftwareSerial gpsSerial(D7, D8); // RX, TX (Connect GPS module accordingly)


// TinyGPS++ object

TinyGPSPlus gps;


// LED & Buzzer Pins

const int ledPin = D1;

const int buzzerPin = D2;


BlynkTimer timer;
```

```
// Function to send GPS data to Blynk
void sendGPSData() {
  if (gps.location.isValid()) {
    float latitude = gps.location.lat();
    float longitude = gps.location.lng();

    Blynk.virtualWrite(V1, latitude);  // Send latitude to Blynk
    Blynk.virtualWrite(V2, longitude); // Send longitude to Blynk
    Blynk.virtualWrite(V3, latitude, longitude); // Update Blynk Map Widget

    Serial.print("Latitude: "); Serial.print(latitude, 6);
    Serial.print(" Longitude: "); Serial.println(longitude, 6);
  }
}

void setup() {
  Serial.begin(115200);
  gpsSerial.begin(9600); // GPS Baud Rate

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  Blynk.begin(auth, ssid, pass);
```

```
    timer.setInterval(2000L, sendGPSData); // Update GPS every 2 seconds
}

void loop() {
    while (gpsSerial.available()) {
        gps.encode(gpsSerial.read());
    }
    Blynk.run();
    timer.run();
}

// Control LED via Blynk (V4)
BLYNK_WRITE(V4) {
    int ledState = param.asInt();
    digitalWrite(ledPin, ledState);
}

// Control Buzzer via Blynk (V5)
BLYNK_WRITE(V5) {
    int buzzerState = param.asInt();
    digitalWrite(buzzerPin, buzzerState);
}
```

SAMPLE INPUT

SAMPLE OUTPUT